



INFORME TÉCNICO · GATE 04

Workspace Structure

Revision Manager

Revisiones CORE gobernadas · edición DRAFT · validación · diff · aprobación · publicación sucesora · rollback lógico

RESULTADO
GATE 04 COMPLETO — implementación, migración PostgreSQL, regresión Gate 03, QA ADMIN/USER y despliegue localhost validados.

Campo	Valor
Aplicación	P&Pmis Construction AI
Repositorio	C:\Users\Ricardo\Documents\GitHub\pypmis-ia-sas
Rama / baseline	main · HEAD dcea01e · cambios Gate 04 sin commit
Fecha de verificación	10 de agosto de 2026 · America/Bogota
Entorno	Docker Compose · localhost · PostgreSQL 16 · Redis 7
Release vigente	ES-PYP-CORE-RECONCILED-20260809 · ID 1 · published
Fuente	PROMPT_04_WORKSPACE_STRUCTURE_REVISION_MANAGER.md
SHA-256 fuente	24542e052d8cc21c0d0e347aa08f12330ba5c802294f978dd6d7f16b6f662575

CONDICIÓN DE QA
La publicación sucesora y el rollback se probaron exclusivamente en una base aislada en memoria. En PostgreSQL local no se creó ningún DRAFT ni se alteró el tenant publicado.

Contenido

N.º	Sección
1	Resumen ejecutivo
2	Alcance, baseline y exclusiones
3	Arquitectura implementada
4	Modelo de revisión
5	Persistencia, migración e inmutabilidad
6	Servicio de dominio y reglas de gobierno
7	API ADMIN
8	RBAC
9	Auditoría
10	UI ADMIN — Revision Manager
11	USER MODE — Enterprise Explorer
12	Flujo Create New Revision
13	Operaciones DRAFT
14	Record Code Preview y recodificación
15	Validate Revision
16	Compare Releases
17	Aprobación explícita
18	Publish Successor y rollback aislado
19	Despliegue localhost
20	Suites y controles de calidad
21	Matriz de aceptación
22	Riesgos y siguiente incremento
23	Anexo A — archivos modificados
24	Anexo B — endpoints, permisos y eventos

LECTURA RECOMENDADA

Las secciones 3 a 9 describen el diseño técnico; 10 a 18 documentan el flujo funcional; 19 a 21 contienen la evidencia de despliegue y aceptación.

1. Resumen ejecutivo

Se implementó el Workspace Structure Revision Manager para administrar cambios sucesores de la estructura empresarial sin editar el release CORE publicado. La solución reutiliza EnterpriseCoreRelease y adopta un snapshot completo editable en estado DRAFT; no introduce un motor genérico de versionado ni duplica físicamente workspaces durante la preparación de la revisión.

El flujo incorpora creación desde el último release vigente, Add/Edit/Move/Classify/Archive, cálculo backend de record_code, recodificación recursiva de descendientes, validación de gobierno, diff detallado, aprobación explícita con hashes, publicación transaccional sucesora, inmutabilidad e idempotencia, y rollback lógico con confirmación.

Control	Estado	Evidencia
---------	--------	-----------

Control	Estado	Evidencia
Create New Revision	PASS	Clone idempotente desde release 1; REV-002 en fixture aislado.
Published release preserved	PASS	PostgreSQL conserva release 1 y fingerprint 58be6cce288d...
Draft editing	PASS	Cinco operaciones DRAFT verificadas.
Validate / Compare / Approval	PASS	Guards de estado y hashes probados.
Publish Successor / Rollback	PASS	Ciclo completo ejecutado en SQLite aislado.
ADMIN / USER QA	PASS	Navegador real sobre localhost; 0 errores de consola.
Tests / Build / Migration	PASS	53 backend + 142 frontend; Alembic 0032 head.

DECISIÓN DE CIERRE

Gate 04 satisface los criterios funcionales y técnicos. Project Creator no fue iniciado, conforme a la exclusión expresa.

2. Alcance, baseline y exclusiones

Baseline preservado

Objeto	Valor verificado
Tenant	P&P Ingeniería y Proyectos · id 1
Release	ES-PYP-CORE-RECONCILED-20260809 · id 1 · published
Workspaces	14
Strategic Objectives	7
Classifications	26
Links	0
PROPERTY / FACILITY	0 / 0
Fingerprint	58be6cce288d4067f83f43bd9ac850da76f20784b4f113aac50398a6bf70c828

Incluido

- Gestión gobernada de revisiones CORE sucesoras en ADMIN MODE.
- Snapshot DRAFT completo, validación, diff, aprobación, publicación e historial.
- Formulario único por tipo y reglas de composición reutilizadas.
- UI accesible con texto e iconos para estados de cambio.
- USER Explorer limitado al release publicado vigente.

Exclusiones respetadas

- Project Creator, EXPERIENCE, PROPERTY, FACILITY y Asset Manager.
- CPM, XML P6, workflow genérico, microservicios o framework universal de versionado.
- Mutación de datos reales para QA funcional del sucesor.

3. Arquitectura implementada

```
React / Vite – WorkspaceRevisionManager
  ↓ REST + Bearer + tenant context
FastAPI – router_admin + exact RBAC guards
  ↓ EnterpriseStructureRevisionService
SQLAlchemy – EnterpriseCoreRelease snapshot DRAFT
  ↓ Alembic 0032 / PostgreSQL trigger
PostgreSQL 16 – release history + workspaces materialized on publish
Redis 7 / Celery – existing platform services, unchanged
```

Capa	Componente	Responsabilidad
Presentación	WorkspaceRevisionManager.tsx	Árbol DRAFT, editor, preview, estados, diff y gate actions.
Integración	enterpriseStructureApi	Contratos tipados para clone, CRUD DRAFT, validate, diff, approve, publish y rollback.
API	router_admin.py	Rutas ADMIN con permiso específico y alcance de organización.
Dominio	revisions.py	Reglas, hashes, snapshot, validación, diff, materialización y auditoría.
Persistencia	EnterpriseCoreRelease	Estado de revisión, snapshot, hashes, aprobación y vínculo al release anterior.
Seguridad	permissions.py + PERMISSION_SEED	Evaluación sin bypass y grants a roles administrativos existentes.
Base de datos	Alembic 20260810_0032	Columnas, FKs, índice y trigger PostgreSQL de inmutabilidad.

La alternativa de snapshot completo fue elegida por simplicidad y compatibilidad. Con 14 workspaces el costo de serialización es bajo, el diff es determinista y la publicación puede materializarse de forma transaccional usando identidades declarativas existentes.

4. Modelo de revisión

Campo	Uso en Gate 04
revision_number	Secuencia lógica; el sucesor del release 1 inicia en 2.
previous_release_id	FK al release base; se conserva después de publicar.
state	draft, published, superseded o unpublished.
snapshot_json	Snapshot editable completo mientras state=draft.
base_content_fingerprint	Fingerprint del release fuente al crear el DRAFT.
content_fingerprint	Hash determinista del release_code y snapshot DRAFT.
validation_json	Resultado, checks, errores, conflictos y hashes validados.
validated_*	Actor, fecha y hash exacto validado.
diff_hash	Hash determinista del cambio contra el release base.
approved_*	Actor, fecha, draft_hash y diff_hash explícitamente aprobados.
created_* / updated_at	Trazabilidad temporal y autoría del release.
published_*	Nulos en DRAFT; obligatorios lógicamente al publicar.

IDENTIDAD

Technical ID y external_key se conservan para nodos existentes. Los nodos nuevos reciben external_key interno no editable y technical_id=None hasta la materialización del publish.

5. Persistencia, migración e inmutabilidad

La migración 20260810_0032 extiende enterprise_core_releases de manera aditiva, backfill-a el release publicado existente y deja published_at/published_by_user_id anulables para soportar DRAFT. No borra tablas ni workspaces.

Control	Implementación	Resultado
Revision metadata	14 columnas nuevas; FKs de created/validated/approved actor.	PASS
Backfill Gate 03	revision_number=id, base fingerprint, created/published actors.	PASS
Published immutability	Listener SQLAlchemy y trigger PostgreSQL.	PASS
Delete protection	Trigger rechaza DELETE físico de releases CORE.	PASS
Draft mutability	Solo OLD.state=draft permite cambiar snapshot y hashes.	PASS
PostgreSQL head	alembic current → 20260810_0032 (head).	PASS

```
20260810_0031 → 20260810_0032 (head)
add governed workspace structure revisions
```

En la base local, después de la migración, existe un único release: ID 1, revision_number 1, state published y conteos 14/7/26/0. No se creó un DRAFT real.

6. Servicio de dominio y reglas de gobierno

Regla	Comportamiento
Fuente vigente	Solo se clona el release published actual; draft previo se devuelve idempotentemente.
Operaciones DRAFT	Add, Edit, Move, Classify y Archive rechazan releases no draft.
Composición	Los hijos permitidos se obtienen del catálogo publicado de workspace types.
Identidad	No existe endpoint para cambiar technical_id ni external_key.
Record Code	Backend calcula el siguiente segmento y recodifica descendientes en movimientos.
Archive	Lógico; bloquea raíz y nodos con hijos activos.
Catálogos	Valida existencia, estado, applicability y required categories.
Hash invalidation	Cada modificación invalida validación y aprobación previas.
Base changed	Publicación aborta con BASE_RELEASE_CHANGED.
Idempotencia	Segundo publish del mismo release retorna el publicado sin mutaciones.

7. API ADMIN

Método	Ruta	Finalidad
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{published_id}/clone	Crear o recuperar DRAFT sucesor.
GET	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}	Consultar release/revisión.
PATCH	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}	Editar nombre del DRAFT.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/record-code-preview	Preview backend BEFORE/AFTER.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/workspaces	Add Workspace.
PATCH	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/workspaces/{key}	Edit Workspace.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/workspaces/{key}/move	Move y recodificación.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/workspaces/{key}/archive	Archive lógico.
PUT	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/workspaces/{key}/classifications	Reemplazar clasificaciones.

Método	Ruta	Finalidad
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/validate	Validar y fijar hashes.
GET	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/diff	Comparar contra previous release.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/approve	Aprobación explícita.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/publish	Publicación sucesora.
POST	/api/v1/admin-configuration/enterprise-structure/enterprise-core-releases/{release_id}/rollback	Rollback lógico confirmado.

SEGURIDAD DE RUTAS

Todas las rutas derivan tenant_id y user_id del contexto autenticado y requieren permiso específico; las mutaciones exigen organization-wide scope.

8. RBAC

Permiso	Operación
admin.enterprise_structure.revision.create	Clone / Create New Revision
admin.enterprise_structure.revision.edit	PATCH, preview y operaciones workspace DRAFT
admin.enterprise_structure.revision.validate	Validate
admin.enterprise_structure.revision.compare	Diff / Compare
admin.enterprise_structure.revision.approve	Approve
admin.enterprise_structure.publish	Publish Successor
admin.enterprise_structure.rollback	Rollback lógico

La base local contiene los siete permisos con dos grants cada uno, correspondientes a los roles administrativos existentes. require_enterprise_permission valida usuario activo, tenant, asignación vigente, permiso y alcance; no existe bypass.

SEPARACIÓN DE FUNCIONES

El modelo permite separar create/edit/validate/compare/approve/publish/rollback. La política local actual concede el conjunto administrativo a organization_admin y configuration_admin; una matriz de segregación más estricta es una decisión de gobierno futura.

9. Auditoría

Evento	Momento
enterprise_structure.revision_created	Creación o clone del DRAFT
enterprise_structure.revision_modified	Add/Edit/Move/Classify/Archive o metadata
enterprise_structure.revision_validated	Resultado, errores, conflictos y hashes
enterprise_structure.revision_approved	Actor y hashes aprobados
enterprise_structure.core_published	Release sucesor materializado
enterprise_structure.core_unpublished	Rollback del release actual

Cada evento se persiste en SecurityEvent con actor, tenant, release, previous release cuando aplica, draft hash, diff hash, timestamp y result. La prueba de rollback exige y encuentra el conjunto completo de seis eventos.

10. UI ADMIN — Revision Manager

La pantalla Enterprise Structure Configuration conserva CompactModuleHeader. Cuando existe un CORE publicado, Revision Manager es la vista inicial; el baseline editable anterior permanece disponible como Published baseline y queda bloqueado por coreLocked.

Zona	Contenido
Release strip	Current Published Release, DRAFT sucesor, Based on y estado.
Gate actions	Validate, Compare, Approve y Publish con habilitación por estado.
Status bar	Draft hash, diff hash, resultado de validación y aprobación.
Workspace tree	Jerarquía DRAFT con Added/Modified/Moved/Archived/Classification/Baseline.
Workspace detail	Record code, tipo, parent, status, cambio, descripción y clasificaciones.
Editor único	Name, Type, Parent, Description, Responsible, Status y classifications.
Move preview	BEFORE / AFTER y lista de descendientes afectados.
Diff	Resumen por acción y tabla detallada con old/new/parents/status/classifications.

Los estados no dependen solo del color: cada nodo muestra texto visible y un icono. Los botones de árbol usan etiquetas accesibles para expandir/contraer.

ADMIN MODE · ENTERPRISE STRUCTURE
Enterprise Structure Configuration
Current Published Release: ES-PYP-CORE-RECONCILED-20260809
[Create New Revision]

11. USER MODE — Enterprise Explorer

Enterprise Explorer no consume el DRAFT. Durante la edición de una revisión continúa mostrando latest_core_release() publicado; después de publicar un sucesor, la misma consulta apunta al nuevo release vigente.

Verificación real	Resultado
Modo	USER MODE · Enterprise Strategy Manager
Submódulo	Enterprise Explorer
Release visible	ES-PYP-CORE-RECONCILED-20260809
Nodos	14
Fingerprint visible	58be6cce288d4067...
DRAFT visible	No
Árbol jerárquico	Sí; record codes 01, 01.01, 01.01.01...
Errores de consola	0

12. Flujo Create New Revision

Published Release 1
ES-PYP-CORE-RECONCILED-20260809
 ↓ Create New Revision
Draft Release 2
ES-PYP-CORE-REV-002
previous_release_id = 1
 ↓ Edit → Validate → Compare → Approve
Publish Successor

1. Confirma que el release seleccionado es published y coincide con el published actual.
2. Devuelve el DRAFT existente si ya fue creado desde la misma base; evita duplicados.
3. Normaliza el snapshot del release 1 y conserva los 14 workspaces lógicos.
4. Genera revision_number 2 y release_code ES-PYP-CORE-REV-002.
5. Registra previous_release_id, base_content_fingerprint, actor y timestamps.
6. Calcula draft_hash y diff_hash inicial; emite revision_created.

RESULTADO AISLADO

El test crea REV-002, conserva las claves ENT-PYP/BU-CORE/PF-ONE/PF-TWO y verifica que el release publicado no cambia.

13. Operaciones DRAFT

Operación	Regla principal
Add Child	Tipo filtrado por allowed_children; external_key generado; technical_id diferido.
Edit	Name, description, responsable y status; no identidad técnica.
Move	Valida ciclos/composición; conserva key/id; recodifica subárbol.
Classify	Reemplaza valores del workspace; duplicados y catálogo controlados.
Archive	Status archived; bloquea raíz o hijos activos; no DELETE físico.

Ejemplo probado

- Add Program ‘AI Program’ bajo PF-ONE con preview 01.01.01.01.
- Edit a ‘AI SaaS Program’ y descripción ‘Controlled draft edit’.
- Classify PF-ONE de growth a operational-excellence.
- Archive del programa nuevo; status archived en DRAFT.
- Intento Portfolio bajo Portfolio rechazado por parent-child incompatibility.

14. Record Code Preview y recodificación

El frontend nunca calcula ni permite digitar record_code. El backend usa el parent, hermanos, sort order y el código excluido cuando se mueve un nodo. El preview se muestra antes de guardar.

Elemento	BEFORE	AFTER
Portfolio PF-ONE	01.01.01	01.02.01
Descendiente Program	01.01.01.01	01.02.01.01
Descendiente Project	01.01.01.01.01	01.02.01.01.01

COBERTURA REFORZADA

La prueba crea explícitamente Portfolio → Program → Project, compara affected_descendants y valida la recodificación persistida de ambos niveles.

15. Validate Revision

Check	Control
single_root	Una raíz exacta
acyclic	Cero ciclos
parent_child_compatible	Composition Rules
record_code_unique	Record codes únicos

Check	Control
external_key_unique	Identidad declarativa única
required_classifications	Categorías obligatorias
category_applicable	Applicability por tipo
links_valid	Links con origen/destino válidos
cross_tenant_zero	Snapshot del tenant actual
no_orphans	Todo parent existe
status_transitions_valid	Estados y archivo válidos
codes_unique	Códigos internos no duplicados

```
VALID
0 errors
0 conflicts
all checks = true
```

El escenario inválido fuerza tenant 999, code y record_code duplicados, un ciclo y un parent inexistente; Validate devuelve valid=false y marca individualmente los checks fallidos.

16. Compare Releases

Acción	Conteo del escenario aislado	Ejemplo
Added	1	Second Business Unit
Modified	1	BU-CORE → Construction Management
Moved	1	PF-ONE bajo la nueva Business Unit
Archived	1	PF-TWO
Classification Changes	1	PF-ONE: growth → operational-excellence
Unchanged	1	Enterprise root

RevisionDiffItem incluye action, old/new record code, type, name, parent before/after, status before/after, classifications before/after y affected descendants. diff_hash se calcula sobre la representación normalizada y ordenada.

17. Aprobación explícita

Validate no implica aprobación. Approve requiere que el DRAFT esté validado, que la validación sea válida y que draft_hash y diff_hash suministrados coincidan con la revisión actual.

Control	Resultado probado
Publish sin aprobación	409 · approval required
Approve con draft hash incorrecto	409 · HASH_MISMATCH
Approve válido	approved_by / approved_at / approved hashes persistidos
Edición posterior	Invalida validation y approval

18. Publish Successor y rollback aislado

Guard de publicación	Comprobación
state=draft	Rechaza releases históricos/no DRAFT.

Guard de publicación	Comprobación
validation=PASS	Mismos draft_hash y diff_hash validados.
approval=PRESENT	Aprobación explícita del contenido actual.
hash=MATCH	Payload, release y aprobación coinciden.
diff_hash=MATCH	Diff no cambió desde aprobación.
source still current	Release base sigue siendo published vigente.

En la fixture aislada, Release 2 pasa a published, Release 1 a superseded y previous_release_id permanece en 1. La materialización actualiza EnterpriseWorkspace por external_key en una transacción. Un segundo publish devuelve el mismo release sin mutaciones.

Rollback exige confirm=true y motivo. El release 2 pasa a unpublished, el release 1 vuelve a published y su snapshot se rematerializa; BU-CORE recupera 'Core Business Unit'. No se elimina ningún release ni workspace.

Control	Estado	Evidencia
Publish successor	PASS	Release 2 published; previous_release_id=1.
Previous preserved	PASS	Release 1 retained as superseded/history.
Second publish	PASS	Safe replay returns same release.
Immutable after publish	PASS	SQLAlchemy raises on field mutation.
BASE_RELEASE_CHANGED	PASS	Competing published release blocks publish.
Logical rollback	PASS	Explicit confirmation restores release 1.

19. Despliegue localhost

Servicio	Endpoint/puerto	Estado
Frontend React/Vite	http://127.0.0.1:5173/app	UP · HTTP 200
FastAPI	http://127.0.0.1:8000	UP · healthy
Readiness	/api/v1/health/ready	api/database/redis = ok
PostgreSQL 16	127.0.0.1:5432	UP · healthy
Redis 7	127.0.0.1:6379	UP · healthy
Celery worker	control-core queue	UP
Celery beat	scheduler	UP

La imagen backend fue reconstruida después de la implementación; Alembic 0032 se aplicó antes de recrear api/worker/beat. Los volúmenes de PostgreSQL y documentos no fueron eliminados.

QA navegador	Evidencia
USER	Enterprise Explorer · release 1 · 14 nodos · no DRAFT.
ADMIN	Enterprise Structure Configuration · CORE published · Create New Revision.
Consola	0 errores.
Datos reales	1 release published; 0 drafts creados por QA.

20. Suites y controles de calidad

Suite/control	Comando o alcance	Resultado
---------------	-------------------	-----------

Suite/control	Comando o alcance	Resultado
Backend Gate 04	pytest test_enterprise_structure_revisions.py	9 passed · 6.72 s
Jerarquía/importador	pytest Gate 2A/importer	20 passed · 23.85 s
Publish/rollback Gate 03	pytest test_enterprise_structure_apply.py	24 passed · 236.37 s
Backend style	ruff format --check + ruff check	118 files · PASS
Frontend focal	Vitest Enterprise + Revision Manager	10 passed
Frontend completo	npm run test -- --run	24 files · 142 passed · 82.52 s
Frontend format	npm run format:check	PASS
Frontend lint	ESLint max-warnings=10	0 errors · 8 warnings preexistentes
Frontend build	tsc && vite build	PASS · 2343 modules
Migration	alembic upgrade/current	0032 head · PostgreSQL PASS
Repository	git diff --check	PASS
Browser QA	USER + ADMIN + console	PASS / PASS / 0 errors

INCIDENTE DE ENTORNO RESUELTO

Una ejecución paralela agotó el espacio temporal de SQLite y bloqueó WSL/Docker. Se eliminaron solo directorios .pytest_tmp del repositorio, se reinició el motor sin borrar volúmenes y las suites se repitieron secuencialmente con 20/20 y 24/24 PASS.

21. Matriz de aceptación

#	Criterio	Estado	Evidencia
1	Crear draft desde published	PASS	REV-002, state=draft
2	Published permanece intacto	PASS	Snapshot/fingerprint preservados
3	previous_release correcto	PASS	previous_release_id=1
4	Add workspace	PASS	AI Program
5	Edit workspace	PASS	AI SaaS Program
6	Move workspace	PASS	PF-ONE movido
7	Archive workspace	PASS	Status archived
8	Classifications	PASS	growth → operational-excellence
9	Record code preview	PASS	01.01.01.01
10	Descendant recoding	PASS	Program + Project recodificados
11	Cycle blocked	PASS	409 cycles
12	Invalid parent blocked	PASS	Composition rule
13	Duplicate record code blocked	PASS	Validate false
14	external_key preserved	PASS	PF-ONE estable
15	Validate valid	PASS	0 errors / 0 conflicts
16	Validate invalid	PASS	5 checks fallidos
17	Diff added	PASS	added=1
18	Diff modified	PASS	modified=1
19	Diff moved	PASS	moved=1
20	Diff archived	PASS	archived=1
21	Diff classifications	PASS	classification_changes=1
22	Approval required	PASS	Publish sin approve bloqueado

#	Criterio	Estado	Evidencia
23	Hash mismatch blocked	PASS	HASH_MISMATCH
24	Base release changed	PASS	BASE_RELEASE_CHANGED
25	Publish successor	PASS	Release 2 published aislado
26	Previous preserved	PASS	Release 1 superseded/histórico
27	Release immutable	PASS	Listener + trigger
28	Second publish safe	PASS	Same release, 0 mutaciones
29	Rollback logical	PASS	Release 1 restored
30	Cross-tenant blocked	PASS	cross_tenant_zero=false
31	RBAC	PASS	7 permisos exactos
32	Audit events	PASS	6 eventos verificados
33	ADMIN UI	PASS	Revision Manager real + tests
34	Regression Gate 03	PASS	44 pruebas históricas
35	Frontend build	PASS	tsc + Vite
36	Migrations PostgreSQL	PASS	Alembic 0032 head

CRITERIOS DE CIERRE

Create New Revision, preservation, editing, preview, validate, compare, approval, publish successor, previous_release, immutability, rollback, ADMIN QA, USER published-only y tests: PASS.

22. Riesgos y siguiente incremento

Riesgo / deuda	Impacto	Mitigación recomendada
Snapshot completo en estructuras muy grandes	Crecimiento de JSON y costo de diff.	Medir con 1k/10k nodos antes de optimizar; no introducir deltas prematuramente.
Segregación de funciones	Dos roles locales poseen approve y publish.	Definir matriz SoD por tenant y asignaciones separadas.
E2E mutacional PostgreSQL	El ciclo completo se ejecutó en fixture aislado.	Añadir base efímera PostgreSQL en CI con clone→publish→rollback.
Concurrencia de drafts	Existe guard de base vigente, no lock de editor multiusuario.	Agregar optimistic version/If-Match si aparece edición concurrente real.
Espacio local limitado	Docker/pytest paralelo puede bloquear WSL.	Mantener >10 GB libres y ejecutar suites pesadas secuencialmente.
8 warnings BIM viewer	No bloquean Gate 04, sí son deuda técnica.	Corregir hooks/dependencias en un sprint de hardening separado.

Recomendación

El siguiente incremento recomendado es un Gate 04 de hardening operativo: E2E transaccional con PostgreSQL efímero, prueba de dos editores concurrentes, segregación approve/publish y medición de snapshots grandes. No iniciar Project Creator automáticamente; ese trabajo requiere una autorización y un prompt propios.

CIERRE

Detener el alcance al finalizar Gate 04. Estado final: completo y desplegado en localhost.

A. Anexo A — archivos modificados

Capa	Archivo
Backend	backend/app/modules/enterprise_structure/constants.py
Backend	backend/app/modules/enterprise_structure/importer/publish.py
Backend	backend/app/modules/enterprise_structure/models.py
Backend	backend/app/modules/enterprise_structure/repository.py
Backend	backend/app/modules/enterprise_structure/revisions.py (nuevo)
Backend	backend/app/modules/enterprise_structure/router_admin.py
Backend	backend/app/modules/enterprise_structure/schemas.py
Backend	backend/app/modules/enterprise_structure/service.py
Backend	backend/alembic/versions/20260810_0032_workspace_revision_manager.py (nuevo)
Backend	backend/tests/test_enterprise_structure_revisions.py (nuevo)
Frontend	frontend/src/features/enterprise-structure/api/index.ts
Frontend	frontend/src/features/enterprise-structure/components/WorkspaceRevisionManager.tsx (nuevo)
Frontend	frontend/src/features/enterprise-structure/enterpriseStructure.css
Frontend	frontend/src/features/enterprise-structure/pages/AdminEnterpriseStructurePage.tsx
Frontend	frontend/src/features/enterprise-structure/pages/EnterpriseExplorerPage.tsx
Frontend	frontend/src/features/enterprise-structure/types/index.ts
Frontend	frontend/tests/workspace-revision-manager.test.tsx (nuevo)

Los artefactos de evidencia del Gate 04 se almacenan en artifacts/enterprise_structure/gate04 y no forman parte de la implementación productiva contabilizada.

B. Anexo B — endpoints, permisos y eventos

Resumen de trazabilidad cruzada:

Flujo	API	Permiso admin.*	Evento enterprise_structure.*
Create	POST .../{published_id}/clone	revision.create	revision_created
Edit/Add/Move/Classify/Archive	PATCH/POST/PUT .../workspaces	revision.edit	revision_modified
Validate	POST .../{id}/validate	revision.validate	revision_validated
Compare	GET .../{id}/diff	revision.compare	—
Approve	POST .../{id}/approve	revision.approve	revision_approved
Publish	POST .../{id}/publish	enterprise_structure.publish	core_published
Rollback	POST .../{id}/rollback	enterprise_structure.rollback	core_unpublished

EVIDENCIA REPRODUCIBLE

Resumen JSON: artifacts\enterprise_structure\gate04\verification_summary.json · Prompt SHA-256 verificado · localhost activo.